



# Manual de Despliegue

Harvist

Versión 1.0.0

Autores: BenjaminDTS · Carlos Vico

Nubium Solutions

Mayo 2026

---

## Manual de Despliegue — Harvist

---

**Versión:** 1.0.0 **Autores:** BenjaminDTS · Carlos Vico (Nubium Solutions) **Última actualización:** Mayo 2026

---

### Tabla de Contenidos

- [Arquitectura del sistema](#1-arquitectura-del-sistema)
- [Requisitos de hardware y software](#2-requisitos-de-hardware-y-software)
- [Preparación del entorno](#3-preparación-del-entorno)

- 4. [Instalación en desarrollo](#4-instalación-en-desarrollo)
- 5. [Variables de entorno](#5-variables-de-entorno)
- 6. [Despliegue en producción](#6-despliegue-en-producción)
- 7. [Despliegue con Docker (recomendado)](#7-despliegue-con-docker-recomendado)
- 8. [Configuración de nginx](#8-configuración-de-nginx)
- 9. [Gestión de procesos con systemd](#9-gestión-de-procesos-con-systemd)
- 10. [Almacenamiento en la nube](#10-almacenamiento-en-la-nube)
- 11. [Monitorización y logs](#11-monitorización-y-logs)
- 12. [Actualizaciones](#12-actualizaciones)
- 13. [Backup y recuperación](#13-backup-y-recuperación)
- 14. [Seguridad en producción](#14-seguridad-en-producción)
- 15. [Solución de problemas](#15-solución-de-problemas)
- 16. [Referencia de variables de entorno](#16-referencia-de-variables-de-entorno)

## 1. Arquitectura del sistema

Harvist está compuesto por cuatro servicios independientes que deben ejecutarse simultáneamente:



| Servicio | Tecnología        | Puerto                 | Rol                                |
|----------|-------------------|------------------------|------------------------------------|
| Frontend | React 18 + Vite   | 5173 (dev) / 80 (prod) | Interfaz de usuario                |
| Backend  | FastAPI + Uvicorn | 8000                   | API REST + WebSocket               |
| Workers  | Celery            | —                      | Procesamiento asíncrono de jobs    |
| Redis    | Redis 7           | 6379                   | Broker de mensajes + caché de jobs |

## 2. Requisitos de hardware y software

### Hardware mínimo (desarrollo)

| Recurso | Mínimo | Recomendado |
|---------|--------|-------------|
|---------|--------|-------------|

|       |         |         |
|-------|---------|---------|
| CPU   | 2 cores | 4 cores |
| RAM   | 4 GB    | 8 GB    |
| Disco | 20 GB   | 50 GB   |
| Red   | 10 Mbps | 50 Mbps |

Hardware recomendado (producción)

| Recurso | Recomendado |
|---------|-------------|
| CPU     | 8 cores     |
| RAM     | 16 GB       |
| Disco   | 200 GB SSD  |
| Red     | 100 Mbps    |

El procesamiento de imágenes con Selenium es CPU-intensivo. Con catálogos grandes (>500 productos), se recomienda al menos 4 cores físicos.

Software requerido

| Software                 | Versión mínima  | Uso                          |
|--------------------------|-----------------|------------------------------|
| Python                   | 3.11+           | Backend + workers            |
| Node.js                  | 18+             | Frontend                     |
| npm                      | 9+              | Gestión de paquetes frontend |
| Docker                   | 24+             | Redis (recomendado)          |
| Docker Compose           | 2.0+            | Orquestación de contenedores |
| Google Chrome / Chromium | Reciente        | Selenium scraping            |
| ChromeDriver             | Matching Chrome | Selenium driver              |

Sistema operativo

- **Linux** (Ubuntu 22.04+ / Debian 12+) — recomendado para producción
- **Windows 11** — soportado para desarrollo
- **macOS** — soportado para desarrollo

## 3. Preparación del entorno

### 3.1 Clonar el repositorio

```
git clone https://github.com/BENJAMINDTS/Harvist.git cd Harvist
```

### 3.2 Instalar Python 3.11+

**Ubuntu/Debian:**

```
sudo apt update sudo apt install python3.11 python3.11-venv python3.11-dev -y python3.11 --version
```

**Windows (usando winget):**

```
winget install Python.Python.3.11
```

### 3.3 Crear entorno virtual

```
python3.11 -m venv .env
```

## Linux/macOS

```
source .env/bin/activate
```

## Windows (PowerShell)

```
.env\Scripts\Activate.ps1
```

### 3.4 Instalar Node.js

**Ubuntu/Debian (via NodeSource):**

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt install nodejs -y node --version npm --version
```

**Windows:**

```
winget install OpenJS.NodeJS.LTS
```

### 3.5 Instalar Chrome/ChromeDriver

## Ubuntu/Debian:

```
# Google Chrome wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | sudo apt-key add - echo
"deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable main" | sudo tee
/etc/apt/sources.list.d/google-chrome.list sudo apt update && sudo apt install google-chrome-stable -y
google-chrome --version
```

## ChromeDriver (automático via undetected-chromedriver)

## No requiere instalación manual – el driver se descarga automáticamente

**Windows:** Descarga Chrome desde <https://www.google.com/chrome/>

## 4. Instalación en desarrollo

### 4.1 Instalar dependencias Python

```
# Activar entorno virtual primero pip install -e ".[dev]"
```

Este comando instala el paquete en modo editable con todas las dependencias de desarrollo (pytest, ruff, pip-audit, etc.).

### 4.2 Instalar dependencias frontend

```
cd frontend npm install cd ..
```

### 4.3 Configurar variables de entorno

```
cp .env.example .env.development
```

Edita `.env.development` con tus valores. Como mínimo, configura:

```
APP_ENV=development SECRET_KEY=tu-clave-secreta-aleatoria-de-32-chars REDIS_URL=redis://localhost:6379/0
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

### 4.4 Levantar Redis con Docker

```
docker compose up -d
```

Verifica que Redis está activo:

```
docker compose ps
```

**Debe mostrar: harvist-redis Up  
0.0.0.0:6379->6379/tcp**

#### 4.5 Arrancar todos los servicios

Abre 4 terminales:

**Terminal 1 — Celery Worker:**

```
# Linux/macOS celery -A workers.celery_app worker --loglevel=info
```

**Windows (requiere --pool=solo)**

```
celery -A workers.celery_app worker --loglevel=info --pool=solo
```

**Terminal 2 — FastAPI Backend:**

```
uvicorn api.main:app --reload --host 0.0.0.0 --port 8000
```

**Terminal 3 — Frontend Vite:**

```
cd frontend npm run dev
```

#### 4.6 Verificar instalación

- Frontend: <http://localhost:5173>
- Backend API: <http://localhost:8000/api/docs> (Swagger UI)
- Health check: <http://localhost:8000/health>

Alternativamente, usa el skill de Claude Code:

```
/launch
```

## 5. Variables de entorno

### 5.1 Ficheros de entorno

Harvist usa ficheros `.env` separados por entorno. Nunca se commitean al repositorio.

| Fichero                       | Entorno               | En Git |
|-------------------------------|-----------------------|--------|
| <code>.env.example</code>     | Plantilla documentada | Sí     |
| <code>.env.development</code> | Desarrollo local      | No     |
| <code>.env.staging</code>     | Pre-producción        | No     |
| <code>.env.production</code>  | Producción            | No     |

El fichero activo se selecciona automáticamente según `APP_ENV`. Al arrancar, Pydantic valida que todas las variables obligatorias estén presentes — si falta alguna, el servidor no arranca y muestra el error exacto.

### 5.2 Crear el fichero de entorno

```
# Desde la raíz del proyecto cp .env.example .env.development
```

En Windows (PowerShell):

```
Copy-Item .env.example .env.development
```

### 5.3 Paso 1 — Configuración base (obligatoria)

Edita `.env.development` y ajusta estos valores primero:

```
# Entorno de ejecucion APP_ENV=development
```

## Desactivar debug en staging/produccion

```
APP_DEBUG=true
```

## Clave secreta – genera un valor aleatorio unico

```
Linux/macOS: python -c "import secrets;  
print(secrets.token_hex(32))"
```

```
Windows PS: python -c "import secrets;  
print(secrets.token_hex(32))"
```

```
SECRET_KEY=pon-aqui-una-clave-aleatoria-de-64-caracteres-hex
```

**URL base de la API (sin barra final)**

```
API_HOST=0.0.0.0 API_PORT=8000 API_PREFIX=/api/v1
```

**Origenes CORS permitidos (separados por coma, sin espacios)**

**En desarrollo: el puerto de Vite**

```
ALLOWED_ORIGINS=http://localhost:5173
```

## 5.4 Paso 2 — Redis y Celery

Si usas el `docker-compose.yml` incluido, los valores por defecto funcionan sin cambio:

```
REDIS_URL=redis://localhost:6379/0 CELERY_BROKER_URL=redis://localhost:6379/0  
CELERY_RESULT_BACKEND=redis://localhost:6379/1
```

Si Redis está en otro host (staging/produccion con Redis gestionado):

```
REDIS_URL=redis://:MI_PASSWORD_REDIS@redis.mi-empresa.com:6379/0  
CELERY_BROKER_URL=redis://:MI_PASSWORD_REDIS@redis.mi-empresa.com:6379/0  
CELERY_RESULT_BACKEND=redis://:MI_PASSWORD_REDIS@redis.mi-empresa.com:6379/1
```

## 5.5 Paso 3 — Navegador para scraping

```
# Tipo de navegador disponible en el servidor BROWSER_TYPE=chrome
```

**En produccion (servidor sin pantalla):  
siempre true**

```
BROWSER_HEADLESS=true
```

**Ruta al binario – dejar vacio para  
auto-deteccion**

```
BROWSER_BINARY_PATH=
```

**Version principal de Chrome (ej. 124).  
Dejar vacio para auto-deteccion**

```
BROWSER_VERSION_MAIN=
```

**Segundos de espera maxima por pagina**

```
BROWSER_TIMEOUT=30
```

**Motor de busqueda de imagenes**

```
SEARCH_ENGINE=bing
```

## Activar resolución EAN -> marca

```
EAN_ENRICHMENT_ENABLED=true
```

Ruta al binario en Windows (ejemplo):

```
BROWSER_BINARY_PATH=C:\Program Files\Google\Chrome\Application\chrome.exe
```

Ruta al binario en Linux (ejemplo):

```
BROWSER_BINARY_PATH=/usr/bin/google-chrome-stable
```

### 5.6 Paso 4 — Descarga de imágenes

```
# Imágenes a descargar por producto (1-10) IMAGES_PER_PRODUCT=3
```

## Dimensiones mínimas para considerar válida una imagen (píxeles)

```
IMAGE_MIN_WIDTH=200 IMAGE_MIN_HEIGHT=200
```

## Redimensionar imagen descargada a este tamaño (0 = no redimensionar)

```
IMAGE_RESIZE_WIDTH=800 IMAGE_RESIZE_HEIGHT=800
```

## Hilos paralelos de descarga (aumentar en servidores con buena red)

DOWNLOAD\_WORKERS=4

## Timeout por descarga individual (segundos)

DOWNLOAD\_TIMEOUT=15

### 5.7 Paso 5 — Almacenamiento

**Opcion A: disco local** (desarrollo y produccion sin S3):

```
STORAGE_BACKEND=local OUTPUT_DIR=imagenes_descargadas FILE_TTL_SECONDS=604800
```

OUTPUT\_DIR es relativo a la raiz del proyecto. En produccion, usar ruta absoluta:

```
OUTPUT_DIR=/var/data/harvist/imagenes
```

**Opcion B: AWS S3** (produccion recomendada):

```
STORAGE_BACKEND=s3 AWS_S3_BUCKET=harvist-prod-images AWS_S3_PREFIX=jobs/ AWS_REGION=eu-west-1  
AWS_ACCESS_KEY_ID=AKIAXXXXXXXXXXXXXXXXX AWS_SECRET_ACCESS_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX  
FILE_TTL_SECONDS=604800
```

**Opcion C: Azure Blob Storage:**

```
STORAGE_BACKEND=azure AZURE_CONTAINER=harvist-images AZURE_BLOB_PREFIX=jobs/  
AZURE_CONNECTION_STRING=DefaultEndpointsProtocol=https;AccountName=...;AccountKey=...;EndpointSuffix=core.windows.net  
FILE_TTL_SECONDS=604800
```

### 5.8 Paso 6 — Proveedor de IA

Necesitas al menos uno. Puedes tener ambos configurados y elegir por job.

**Opcion A: Groq** (mas rapido, coste bajo):

```
AI_PROVIDER=groq GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX  
GROQ_MODEL=llama-3.3-70b-versatile
```

Obtener API key gratuita en: <https://console.groq.com>

**Opcion B: Anthropic Claude** (mayor calidad de texto):

```
ENABLE_AI_DESCRIPTIONS=true CLAUDE_API_KEY=sk-ant-api03-XXXXXXXXXXXXXXXXXXXXXXXXXXXX  
CLAUDE_MODEL=claude-sonnet-4-6 CLAUDE_MAX_TOKENS=2048 CLAUDE_TIMEOUT=60 CLAUDE_MAX_RETRIES=3  
CLAUDE_BATCH_SIZE=10 CLAUDE_STORE_TYPE=file CLAUDE_PROMPT_FILE=prompts/descripcion_producto.txt
```

Obtener API key en: <https://console.anthropic.com>

### 5.9 Paso 7 — Integraciones ERP/CMS (opcionales)

Solo configura las integraciones que vayas a usar.

#### Dolibarr:

```
DOLIBARR_URL=https://dolibarr.mi-empresa.com DOLIBARR_API_KEY=abcdef1234567890abcdef1234567890
```

## Solo si necesitas campos extra via base de datos directa

```
DOLIBARR_DB_HOST=localhost DOLIBARR_DB_PORT=3306 DOLIBARR_DB_NAME=dolibarr DOLIBARR_DB_USER=dolibarr_user  
DOLIBARR_DB_PASS=mi_password_segura DOLIBARR_DB_PREFIX=llx_
```

Para obtener la API key de Dolibarr: Configuración > Seguridad > Tokens de API.

#### Odoo:

```
ODOO_URL=https://odoo.mi-empresa.com ODOO_DB=mi_base_de_datos ODOO_USER=admin@mi-empresa.com  
ODOO_PASSWORD=mi_password_odoo
```

#### WordPress / WooCommerce:

```
WORDPRESS_URL=https://mi-tienda.com WORDPRESS_CONSUMER_KEY=ck_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX  
WORDPRESS_CONSUMER_SECRET=cs_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX WORDPRESS_APP_USER=admin  
WORDPRESS_APP_PASSWORD=xxxx xxxx xxxx xxxx xxxx xxxx
```

## Solo si usas acceso directo a MySQL

```
WORDPRESS_DB_HOST=localhost WORDPRESS_DB_PORT=3306 WORDPRESS_DB_NAME=wordpress WORDPRESS_DB_USER=wp_user  
  
WORDPRESS_DB_PASS=mi_password_mysql WORDPRESS_DB_PREFIX=wp_
```

## Rutas a los ficheros de configuración persistentes

```
WP_CONFIG_PATH=data/wp_config.json WP_DB_CONFIG_PATH=data/wp_db_config.json
```

Para generar las claves WooCommerce: WooCommerce > Ajustes > Avanzado > REST API > Agregar clave (permisos: Lectura/Escritura).

Para generar contraseña de aplicación WordPress: Usuarios > Tu perfil > Contraseñas de aplicación.

## 5.10 Paso 8 — Logs

```
LOG_LEVEL=DEBUG LOG_DIR=logs LOG_ROTATION=100 MB LOG_RETENTION=30 days
```

En produccion:

```
LOG_LEVEL=INFO LOG_DIR=/var/log/harvist
```

## 5.11 Paso 9 — Resolucion de marcas (opcional)

```
# Proxy rotativo para evitar bloqueos (opcional, dejar vacio si no tienes) ROTATING_PROXY_URL=
```

# Timeouts para peticiones HTTP de marcas (segundos)

```
BRAND_HTTP_TIMEOUT=10 AMAZON_HTTP_TIMEOUT=15
```

# Ruta al fichero de cache de prefijos GS1

```
BRAND_CACHE_PATH=data/brand_cache.json
```

# Horas antes de que el worker de limpieza borre imagenes candidatas huérfanas

```
CANDIDATES_TTL_HOURS=24
```

## 5.12 Verificar configuracion

Tras editar el `.env`, verifica que Pydantic puede cargarlo sin errores:

```
python -c "from api.core.config import get_settings; s = get_settings(); print('OK -', s.app_env)"
```

Si hay variable obligatoria sin valor, el comando muestra el campo exacto que falta.

## 6. Despliegue en producción

### 6.1 Preparar el servidor

```
# Actualizar paquetes sudo apt update && sudo apt upgrade -y
```

## Instalar dependencias del sistema

```
sudo apt install -y git python3.11 python3.11-venv nodejs npm \ google-chrome-stable nginx certbot  
  
python3-certbot-nginx \ redis-server
```

## Crear usuario de servicio (no usar root)

```
sudo useradd -m -s /bin/bash harvist sudo usermod -aG docker harvist
```

### 6.2 Clonar y configurar

```
sudo su - harvist git clone https://github.com/BENJAMINDTS/Harvist.git /opt/harvist cd /opt/harvist
```

```
python3.11 -m venv .venv  
source .venv/bin/activate  
pip install -e "." # Sin [dev] en producción
```

```
cd frontend && npm install && npm run build && cd ..
```

### 6.3 Configurar variables de producción

```
cp .env.example .env.production nano .env.production
```

Valores clave para producción:

```
APP_ENV=production APP_DEBUG=false SECRET_KEY=<genera con: python -c "import secrets;  
print(secrets.token_hex(32))">
```

## CORS: solo tu dominio

```
ALLOWED_ORIGINS=https://harvist.tuempresa.com
```

## Redis externo (si no usas el local)

```
REDIS_URL=redis://redis-host:6379/0
```

## HTTPS

```
API_HOST=0.0.0.0 API_PORT=8000
```

## Logs

```
LOG_LEVEL=INFO LOG_DIR=/var/log/harvist LOG_ROTATION=100 MB LOG_RETENTION=30 days
```

## Almacenamiento: S3 en producción

```
STORAGE_BACKEND=s3 AWS_S3_BUCKET=harvist-prod-images
```

### 6.4 Construir el frontend para producción

```
cd /opt/harvist/frontend npm run build
```

## Genera: frontend/dist/

## 7. Despliegue con Docker (recomendado)

Para producción, se recomienda usar Docker Compose completo con todos los servicios.

## 7.1 docker-compose.prod.yml

Crea el fichero `/opt/harvist/docker-compose.prod.yml`:

```
version: '3.8'

services:
  redis:
    image: redis:7-alpine
    container_name: harvist-redis
    restart: unless-stopped
    ports:
      - "127.0.0.1:6379:6379"

  backend:
    image: harvist/backend
    container_name: harvist-backend
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:8080"

volumes:
  redis-data:
    driver: local
    driver_opts:
      type: none
      device: /data/redis

command: redis-server --save 60 1 --appendonly yes --requirepass ${REDIS_PASSWORD}
healthcheck:
  test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5
```

- "127.0.0.1:8000:8000"

env\_file:

- .env.production

volumes:

- ./imagenes\_descargadas:/app/imagenes\_descargadas
- ./data:/app/data
- ./logs:/app/logs

depends\_on:

redis:

condition: service\_healthy

command: uvicorn api.main:app --host 0.0.0.0 --port 8000 --workers 4

```
worker:
build:
context: .
dockerfile: Dockerfile
container_name: harvist-worker
restart: unless-stopped
env_file:
```

- .env.production

```
volumes:
```

- ./imagenes\_descargadas:/app/imagenes\_descargadas
- ./data:/app/data
- ./logs:/app/logs

```
depends_on:
redis:
condition: service_healthy
command: celery -A workers.celery_app worker --loglevel=info --concurrency=2
```

```
volumes:
redis-data:
driver: local
```

## 7.2 Dockerfile

Crea `/opt/harvist/Dockerfile`:

```
FROM python:3.11-slim
```

## Instalar Chrome y dependencias del sistema

```
RUN apt-get update && apt-get install -y \ wget gnupg2 curl \ google-chrome-stable \  
  
--no-install-recommends && \ rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
```

## Copiar e instalar dependencias Python

```
COPY pyproject.toml ./ RUN pip install --no-cache-dir -e ". "
```

## Copiar código fuente

```
COPY api/ ./api/ COPY services/ ./services/ COPY workers/ ./workers/
```

## Crear directorios necesarios

```
RUN mkdir -p imagenes_descargadas data logs
```

```
EXPOSE 8000
```

## 7.3 Arrancar en producción

```
cd /opt/harvist
```

# Cargar variables

```
export $(grep -v '^#' .env.production | xargs)
```

# Levantar todos los servicios

```
docker compose -f docker-compose.prod.yml up -d
```

# Ver logs

```
docker compose -f docker-compose.prod.yml logs -f backend docker compose -f docker-compose.prod.yml logs -f worker
```

## 8. Configuración de nginx

nginx actúa como proxy inverso para el backend (API) y sirve el frontend (estático).

### 8.1 Obtener certificado SSL

```
sudo certbot --nginx -d harvist.tuempresa.com
```

### 8.2 Configuración nginx

Crea `/etc/nginx/sites-available/harvist`:

```
# Redirigir HTTP → HTTPS server { listen 80; server_name harvist.tuempresa.com; return 301 https://$host$request_uri; }
```

```
server {  
    listen 443 ssl http2;  
    server_name harvist.tuempresa.com;
```

```
    ssl_certificate /etc/letsencrypt/live/harvist.tuempresa.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/harvist.tuempresa.com/privkey.pem;  
    ssl_protocols TLSv1.2 TLSv1.3;
```

```
# Cabeceras de seguridad
add_header X-Content-Type-Options nosniff;
add_header X-Frame-Options DENY;
add_header X-XSS-Protection "1; mode=block";
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains";
```

```
# Frontend (estático)
root /opt/harvist/frontend/dist;
index index.html;
```

```
location / {
    try_files $uri $uri/ /index.html;
}
```

```
# API Backend
location /api/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 300s;
    proxy_connect_timeout 10s;
}
```

```
# WebSocket (progreso en tiempo real)
location /api/v1/jobs/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_read_timeout 3600s;
}
```

```
# Swagger UI (opcional – deshabilitar en producción pública)
location /api/docs {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    # Restringir acceso por IP en producción:
    # allow 192.168.1.0/24;
    # deny all;
}
}
```

### 8.3 Activar sitio

```
sudo ln -s /etc/nginx/sites-available/harvist /etc/nginx/sites-enabled/ sudo nginx -t sudo systemctl
reload nginx
```

## 9. Gestión de procesos con systemd

Para que los servicios arranquen automáticamente con el servidor:

## 9.1 Servicio Backend FastAPI

Crea `/etc/systemd/system/harvist-backend.service`:

```
[Unit] Description=Harvist FastAPI Backend After=network.target redis.service Requires=redis.service
```

```
[Service]
Type=exec
User=harvist
WorkingDirectory=/opt/harvist
EnvironmentFile=/opt/harvist/.env.production
ExecStart=/opt/harvist/.venv/bin/uvicorn api.main:app \
--host 0.0.0.0 \
--port 8000 \
--workers 4 \
--access-log
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

## 9.2 Servicio Celery Worker

Crea `/etc/systemd/system/harvist-worker.service`:

```
[Unit] Description=Harvist Celery Worker After=network.target redis.service Requires=redis.service
```

```
[Service]
Type=exec
User=harvist
WorkingDirectory=/opt/harvist
EnvironmentFile=/opt/harvist/.env.production
ExecStart=/opt/harvist/.venv/bin/celery \
-A workers.celery_app worker \
--loglevel=info \
--concurrency=2
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

## 9.3 Activar servicios

```
sudo systemctl daemon-reload sudo systemctl enable harvist-backend harvist-worker sudo systemctl start
harvist-backend harvist-worker
```

# Verificar estado

```
sudo systemctl status harvist-backend sudo systemctl status harvist-worker
```

## 10. Almacenamiento en la nube

Para producción con alto volumen de imágenes, usa S3 o Azure Blob.

### 10.1 AWS S3

Crear bucket:

```
aws s3 mb s3://harvist-prod-images --region eu-west-1
```

Política de bucket (solo acceso desde la aplicación):

```
{ "Version": "2012-10-17", "Statement": [ { "Effect": "Allow", "Principal": {"AWS":  
"arn:aws:iam::CUENTA:user/harvist-app"}, "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject",  
"s3:ListBucket"], "Resource": [ "arn:aws:s3::harvist-prod-images", "arn:aws:s3::harvist-prod-images/*" ]  
} ] }
```

Variables de entorno:

```
STORAGE_BACKEND=s3 AWS_S3_BUCKET=harvist-prod-images AWS_S3_PREFIX=jobs/ AWS_REGION=eu-west-1  
AWS_ACCESS_KEY_ID=AKIAXXXXXXXXXXXXXXXXX AWS_SECRET_ACCESS_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

### 10.2 Azure Blob Storage

Crear contenedor:

```
az storage container create --name harvist-images --account-name miaccount
```

Variables de entorno:

```
STORAGE_BACKEND=azure AZURE_CONTAINER=harvist-images AZURE_BLOB_PREFIX=jobs/  
AZURE_CONNECTION_STRING=DefaultEndpointsProtocol=https;AccountName=...
```

## 11. Monitorización y logs

### 11.1 Estructura de logs

Los logs se generan en formato JSON estructurado en `/var/log/harvist/`:

```
/var/log/harvist/ ■■■ harvist_YYYY-MM-DD.log # Log diario rotado ■■■ error_YYYY-MM-DD.log # Solo errores
```

Configurar en `.env.production`:

```
LOG_LEVEL=INFO
LOG_DIR=/var/log/harvist
LOG_ROTATION=100 MB
LOG_RETENTION=30 days
```

## 11.2 Ver logs en tiempo real

```
# Backend sudo journalctl -u harvist-backend -f
```

## Worker

```
sudo journalctl -u harvist-worker -f
```

## Logs de aplicación

```
tail -f /var/log/harvist/harvist_$(date +%Y-%m-%d).log
```

## 11.3 Monitorizar Redis

```
# CLI de Redis redis-cli monitor
```

## Información general

```
redis-cli info
```

## Colas Celery pendientes

```
redis-cli llen celery
```

## 11.4 Estado de Celery

```
# Lista de workers activos celery -A workers.celery_app inspect active
```

## Tareas en cola

```
celery -A workers.celery_app inspect reserved
```

## Estadísticas

```
celery -A workers.celery_app inspect stats
```

### 11.5 Health check

```
curl http://localhost:8000/health
```

**Respuesta esperada: {"status": "ok",  
"env": "production"}**

## 12. Actualizaciones

### 12.1 Procedimiento de actualización

```
# 1. Activar entorno virtual cd /opt/harvist source .venv/bin/activate
```

## 2. Obtener cambios

```
git pull origin main
```

## 3. Actualizar dependencias Python

```
pip install -e "."
```

## 4. Actualizar dependencias y construir frontend

```
cd frontend npm install npm run build cd ..
```

## 5. Reiniciar servicios

```
sudo systemctl restart harvist-backend sudo systemctl restart harvist-worker
```

## 6. Verificar

```
curl http://localhost:8000/health sudo systemctl status harvist-backend harvist-worker
```

### 12.2 Con Docker

```
cd /opt/harvist
```

## Rebuild imágenes

```
docker compose -f docker-compose.prod.yml build
```

## Reiniciar con zero-downtime

```
docker compose -f docker-compose.prod.yml up -d --no-deps backend worker
```

### 12.3 Migraciones de datos

Actualmente Harvist no usa una base de datos relacional propia (los datos de jobs se almacenan en Redis con TTL). Si una actualización requiere migración de caché:

```
# Limpiar caché de jobs viejos (opcional) redis-cli keys "job:*" | xargs redis-cli del
```

Los datos de brand\_cache.json persisten entre versiones

No borrar salvo que se indique explícitamente en las notas de release

## 13. Backup y recuperación

### 13.1 Qué hacer backup

| Dato                     | Ubicación                             | Frecuencia     |
|--------------------------|---------------------------------------|----------------|
| Variables de entorno     | .env.production                       | En cada cambio |
| Caché de marcas          | data/brand_cache.json                 | Diario         |
| Configuración WP         | data/wp_config.json                   | En cada cambio |
| Imágenes de jobs activos | imagenes_descargadas/                 | Diario         |
| Configuración nginx      | /etc/nginx/sites-available/           | En cada cambio |
| Scripts systemd          | /etc/systemd/system/harvist-*.service | En cada cambio |

### 13.2 Script de backup

```
#!/bin/bash
```

/opt/harvist/scripts/backup.sh

```
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="/backups/harvist/$DATE"
mkdir -p "$BACKUP_DIR"
```

## Caché de marcas (el dato más valioso)

```
cp /opt/harvist/data/brand_cache.json "$BACKUP_DIR/"
```

## Configuraciones persistidas

```
cp /opt/harvist/data/wp_config.json "$BACKUP_DIR/" 2>/dev/null || true
```

## Variables de entorno (sin subir a repositorios)

```
cp /opt/harvist/.env.production "$BACKUP_DIR/env.backup"
```

## Redis dump (snapshot)

```
redis-cli BGSAVE sleep 2 cp /var/lib/redis/dump.rdb "$BACKUP_DIR/redis.rdb"
```

```
echo "Backup completado en $BACKUP_DIR"
```

Añadir al cron del sistema:

```
crontab -e
```

## Añadir:

```
0 2 * /opt/harvist/scripts/backup.sh >> /var/log/harvist-backup.log 2>&1
```

### 13.3 Recuperación

```
# Restaurar caché de marcas cp /backups/harvist/FECHA/brand_cache.json /opt/harvist/data/
```

## Restaurar Redis

```
sudo systemctl stop redis sudo cp /backups/harvist/FECHA/redis.rdb /var/lib/redis/dump.rdb sudo chown  
redis:redis /var/lib/redis/dump.rdb sudo systemctl start redis
```

## 14. Seguridad en producción

### 14.1 Lista de verificación

- [ ] `APP_ENV=production` y `APP_DEBUG=false`
- [ ] `SECRET_KEY` es aleatoria y tiene al menos 32 caracteres
- [ ] `ALLOWED_ORIGINS` solo contiene tu dominio (nunca \*)
- [ ] Redis protegido con contraseña (`REDIS_PASSWORD`)
- [ ] Redis solo accesible en localhost (`127.0.0.1:6379`)
- [ ] HTTPS habilitado con certificado válido
- [ ] Swagger UI (`/api/docs`) restringido por IP o deshabilitado
- [ ] Firewall: solo puertos 80, 443 y 22 abiertos al exterior
- [ ] Usuario de servicio `harvist` sin sudo
- [ ] Logs sin datos sensibles (keys, tokens, passwords)
- [ ] Dependencias auditadas: `pip-audit`
- [ ] Chrome ejecutado en modo headless: `BROWSER_HEADLESS=true`

### 14.2 Auditoría de dependencias

```
# Python source .venv/bin/activate pip-audit
```

## Frontend

```
cd frontend npm audit
```

## 14.3 Configuración de firewall (UFW)

```
sudo ufw allow 22/tcp # SSH sudo ufw allow 80/tcp # HTTP sudo ufw allow 443/tcp # HTTPS sudo ufw deny 8000  
# Bloquear acceso directo al backend sudo ufw deny 6379 # Bloquear acceso externo a Redis sudo ufw enable
```

## 14.4 Rate limiting

El rate limiting está configurado en la API:

- Jobs: máximo 10 por minuto por IP

Para ajustarlo, modifica `api/core/security.py` o usa las variables de entorno correspondientes.

## 15. Solución de problemas

### El backend no arranca

```
# Ver error exacto sudo journalctl -u harvist-backend -n 50 --no-pager
```

## Comprobar variables de entorno

```
cd /opt/harvist source .venv/bin/activate python -c "from api.core.config import get_settings; s =  
get_settings(); print('OK')"
```

## Probar arranque manual

```
uvicorn api.main:app --host 0.0.0.0 --port 8000
```

### El worker Celery no procesa jobs

```
# Verificar conexión a Redis redis-cli ping # Debe responder: PONG
```

## Ver cola de tareas pendientes

```
redis-cli llen celery
```

## Reiniciar worker

```
sudo systemctl restart harvist-worker sudo journalctl -u harvist-worker -f
```

### Chrome/Selenium falla al arrancar

```
# Verificar que Chrome está instalado google-chrome --version
```

## Probar en modo no headless para depurar

```
BROWSER_HEADLESS=false uvicorn api.main:app --reload
```

## En servidor sin pantalla, instalar Xvfb

```
sudo apt install xvfb export DISPLAY=:99 Xvfb :99 -screen 0 1920x1080x24 &
```

### Error "GROQ\_API\_KEY not configured"

```
# Verificar que la variable está en el fichero .env grep GROQ_API_KEY .env.development
```

## Verificar que el fichero se carga

```
python -c "from api.core.config import get_settings; print(get_settings().groq_api_key)"
```

### El frontend no conecta con la API

```
# Verificar que el backend está corriendo curl http://localhost:8000/health
```

## Verificar proxy nginx (producción)

```
sudo nginx -t sudo systemctl status nginx
```

## Verificar CORS

```
grep ALLOWED_ORIGINS .env.production
```

Jobs quedan en estado PENDIENTE indefinidamente

```
# El worker no está corriendo o no ve Redis sudo systemctl status harvist-worker
```

## Ver si hay workers activos

```
celery -A workers.celery_app inspect active
```

## Limpiar tareas bloqueadas (con cuidado)

```
celery -A workers.celery_app purge
```

Disco lleno por imágenes

```
# Ver uso de disco df -h du -sh /opt/harvist/imagenes_descargadas/
```

## Limpiar jobs caducados manualmente

```
find /opt/harvist/imagenes_descargadas -maxdepth 1 -type d -mtime +7 -exec rm -rf {} \;
```

## 16. Referencia de variables de entorno

Referencia completa de todas las variables disponibles en `.env.example`:

### Aplicación

| Variable        | Tipo | Default               | Descripción                       |
|-----------------|------|-----------------------|-----------------------------------|
| APP_ENV         | str  | development           | Entorno activo                    |
| APP_DEBUG       | bool | false                 | Modo debug (nunca true en prod)   |
| SECRET_KEY      | str  | —                     | Clave secreta de la aplicación    |
| API_HOST        | str  | 0.0.0.0               | Host de binding del servidor      |
| API_PORT        | int  | 8000                  | Puerto del servidor               |
| API_PREFIX      | str  | /api/v1               | Prefijo de rutas API              |
| ALLOWED_ORIGINS | str  | http://localhost:5173 | CORS origins (separados por coma) |

### Redis / Celery

| Variable              | Tipo | Default                  | Descripción           |
|-----------------------|------|--------------------------|-----------------------|
| REDIS_URL             | str  | redis://localhost:6379/0 | URL de Redis          |
| CELERY_BROKER_URL     | str  | redis://localhost:6379/0 | Broker de Celery      |
| CELERY_RESULT_BACKEND | str  | redis://localhost:6379/1 | Backend de resultados |

### Navegador (Selenium)

| Variable             | Tipo | Default | Descripción                              |
|----------------------|------|---------|------------------------------------------|
| BROWSER_TYPE         | enum | chrome  | chrome / opera / edge / brave / chromium |
| BROWSER_BINARY_PATH  | str  | —       | Ruta al binario del navegador            |
| BROWSER_HEADLESS     | bool | true    | Modo sin interfaz gráfica                |
| BROWSER_VERSION_MAIN | int  | —       | Versión principal de Chrome              |

|                 |     |    |                               |
|-----------------|-----|----|-------------------------------|
| BROWSER_TIMEOUT | int | 30 | Timeout por página (segundos) |
|-----------------|-----|----|-------------------------------|

### Búsqueda

| Variable               | Tipo | Default | Descripción                          |
|------------------------|------|---------|--------------------------------------|
| SEARCH_ENGINE          | enum | bing    | bing / google / duckduckgo           |
| EAN_ENRICHMENT_ENABLED | bool | true    | Activar resolución de marcas por EAN |

### Descarga de imágenes

| Variable            | Tipo | Default | Descripción                        |
|---------------------|------|---------|------------------------------------|
| IMAGES_PER_PRODUCT  | int  | 3       | Imágenes a descargar por producto  |
| IMAGE_MIN_WIDTH     | int  | 200     | Ancho mínimo de imagen válida (px) |
| IMAGE_MIN_HEIGHT    | int  | 200     | Alto mínimo de imagen válida (px)  |
| IMAGE_RESIZE_WIDTH  | int  | 800     | Ancho al redimensionar             |
| IMAGE_RESIZE_HEIGHT | int  | 800     | Alto al redimensionar              |
| DOWNLOAD_WORKERS    | int  | 4       | Hilos paralelos de descarga        |
| DOWNLOAD_TIMEOUT    | int  | 15      | Timeout por descarga (segundos)    |

### Almacenamiento

| Variable         | Tipo | Default              | Descripción                   |
|------------------|------|----------------------|-------------------------------|
| STORAGE_BACKEND  | enum | local                | local / s3 / azure            |
| OUTPUT_DIR       | str  | imagenes_descargadas | Directorio local de salida    |
| FILE_TTL_SECONDS | int  | 604800               | Vigencia de ficheros (7 días) |

### AWS S3

| Variable | Tipo | Descripción |
|----------|------|-------------|
|----------|------|-------------|

|                       |     |                              |
|-----------------------|-----|------------------------------|
| AWS_S3_BUCKET         | str | Nombre del bucket S3         |
| AWS_S3_PREFIX         | str | Prefijo de ruta en el bucket |
| AWS_REGION            | str | Región AWS                   |
| AWS_ACCESS_KEY_ID     | str | Clave de acceso IAM          |
| AWS_SECRET_ACCESS_KEY | str | Secreto IAM                  |

Azure Blob

| Variable                | Tipo | Descripción                      |
|-------------------------|------|----------------------------------|
| AZURE_CONTAINER         | str  | Nombre del contenedor            |
| AZURE_BLOB_PREFIX       | str  | Prefijo de ruta en el contenedor |
| AZURE_CONNECTION_STRING | str  | Cadena de conexión de Azure      |

Logging

| Variable      | Tipo | Default | Descripción                 |
|---------------|------|---------|-----------------------------|
| LOG_LEVEL     | enum | INFO    | ERROR / WARN / INFO / DEBUG |
| LOG_DIR       | str  | logs    | Directorio de logs          |
| LOG_ROTATION  | str  | 100 MB  | Tamaño máximo por fichero   |
| LOG_RETENTION | str  | 30 days | Tiempo de retención de logs |

IA — Groq

| Variable     | Tipo | Descripción                             |
|--------------|------|-----------------------------------------|
| AI_PROVIDER  | enum | groq / claude                           |
| GROQ_API_KEY | str  | API key de Groq                         |
| GROQ_MODEL   | str  | Modelo a usar (llama-3.3-70b-versatile) |

IA — Claude (Anthropic)

| Variable | Tipo | Default | Descripción |
|----------|------|---------|-------------|
|----------|------|---------|-------------|

|                        |      |                   |                                |
|------------------------|------|-------------------|--------------------------------|
| ENABLE_AI_DESCRIPTIONS | bool | false             | Activar generación con Claude  |
| CLAUDE_API_KEY         | str  | —                 | API key de Anthropic           |
| CLAUDE_MODEL           | str  | claude-sonnet-4-6 | Modelo Claude a usar           |
| CLAUDE_MAX_TOKENS      | int  | 2048              | Máximo de tokens por respuesta |
| CLAUDE_TIMEOUT         | int  | 60                | Timeout por llamada (segundos) |
| CLAUDE_MAX_RETRIES     | int  | 3                 | Reintentos en caso de error    |
| CLAUDE_BATCH_SIZE      | int  | 10                | Productos por lote             |

Resolución de marcas

| Variable             | Tipo | Descripción                                  |
|----------------------|------|----------------------------------------------|
| ROTATING_PROXY_URL   | str  | Proxy rotativo para scraping (opcional)      |
| BRAND_HTTP_TIMEOUT   | int  | Timeout para peticiones de marcas (segundos) |
| AMAZON_HTTP_TIMEOUT  | int  | Timeout específico para Amazon (segundos)    |
| BRAND_CACHE_PATH     | str  | Ruta al fichero brand_cache.json             |
| CANDIDATES_TTL_HOURS | int  | Horas hasta limpiar imágenes candidatas      |

Integraciones ERP/CMS

| Variable         | Descripción                                       |
|------------------|---------------------------------------------------|
| DOLIBARR_URL     | URL base de la instancia Dolibarr                 |
| DOLIBARR_API_KEY | Clave DOLAPIKEY                                   |
| DOLIBARR_DB_HOST | Host MySQL de Dolibarr (para campos extra vía BD) |
| DOLIBARR_DB_PORT | Puerto MySQL                                      |
| DOLIBARR_DB_NAME | Nombre de la base de datos                        |
| DOLIBARR_DB_USER | Usuario MySQL                                     |

|                                        |                                                 |
|----------------------------------------|-------------------------------------------------|
| <code>DOLIBARR_DB_PASS</code>          | Contraseña MySQL                                |
| <code>DOLIBARR_DB_PREFIX</code>        | Prefijo de tablas (default: <code>llx_</code> ) |
| <code>ODOO_URL</code>                  | URL base de la instancia Odoo                   |
| <code>ODOO_DB</code>                   | Nombre de la base de datos Odoo                 |
| <code>ODOO_USER</code>                 | Email del usuario Odoo                          |
| <code>ODOO_PASSWORD</code>             | Contraseña o API key de Odoo                    |
| <code>WORDPRESS_URL</code>             | URL base de WordPress                           |
| <code>WORDPRESS_CONSUMER_KEY</code>    | Consumer Key de WooCommerce REST API            |
| <code>WORDPRESS_CONSUMER_SECRET</code> | Consumer Secret de WooCommerce REST API         |
| <code>WORDPRESS_APP_USER</code>        | Usuario de WordPress                            |
| <code>WORDPRESS_APP_PASSWORD</code>    | Contraseña de aplicación de WordPress           |
| <code>WORDPRESS_DB_HOST</code>         | Host MySQL de WordPress                         |
| <code>WORDPRESS_DB_PORT</code>         | Puerto MySQL                                    |
| <code>WORDPRESS_DB_NAME</code>         | Nombre de base de datos WordPress               |
| <code>WORDPRESS_DB_USER</code>         | Usuario MySQL                                   |
| <code>WORDPRESS_DB_PASS</code>         | Contraseña MySQL                                |
| <code>WP_CONFIG_PATH</code>            | Ruta al fichero de config REST persistida       |
| <code>WP_DB_CONFIG_PATH</code>         | Ruta al fichero de config MySQL persistida      |